Password Guessing Based on Deep Learning

Tianrui Gu

Abstract

Password is the most commonly used authentication method. But password is easy to be attacked. Attackers can guess users' passwords by generating some passwords that are similar to the leaked passwords. It is necessary to tell users how to avoid being attacked. Since password strength is a statistic that reflects password resistance to guessing attacks, the first step is to properly estimate password strength through password guessing attacks. With the development of deep learning, we can do this efficiently. In this research, I use generative adversarial network(GAN) to guess the password. Compared to the former guessing model, I not only use WGAN-GP, but also optimizing the structure of the original algorithm initiated by Hitaj[1]. This research introduced the theory of GAN, WGAN and WGAN-GP respectively, proved that GAN do has a strong ability to guess the password, which will change the information security in the community. And this research provides a tip for training GAN: early stopping. Also, this research proposed some methods for users to avoid the attack.

Keywords：Password Guessing, Deep Learning, Generative Adversarial Network

1. Introduction

Password is the most commonly used authentication method at present. Usually, users' password settings follow certain rules, which provides an opportunity for password cracking.

Passwords' popularity is mainly due to their unparalleled deployability and usability.[2].But they has a security flaw. They are easy to be attacked.

The recent massive password leaks on multiple platforms around the world have been a wake-up call for researchers and stakeholders. Therefore, it becomes more critical to help users build stronger passwords. Since password strength is a statistic that reflects password resistance to guessing attacks, the first step is to properly estimate password strength through password guessing attacks.[1]
Password guessing is a method to crack the user's password, which matches the user's real password by generating a burst password set in order to achieve the purpose of password cracking.

Most of the traditional password guessing methods are based on Markov model and rule-based model. They have the disadvantages of taking a long time, occupying a large amount of storage resources, and the generated passwords are limited to the artificial password structure.

With the growth of hardware computing power, the use of deep learning for password guessing has also become a new development direction, because the traditional password guessing methods are mostly based on specific rules and probability statistics. It may not be possible to have comprehensive data to reflect the distribution characteristics of export orders. The strong representation ability of the deep learning model provides the possibility to solve this problem. It can explore a larger password space, and the generated passwords are not constrained by training data and artificial rules. Based on the generative adversarial network (GAN), people use the method of deep learning to guess the password efficiently. GAN has many variants: DCGAN, WGAN, WGAN-GP, Adversarial Autoencoder (AAE)…

In this study, I use IWGAN(WGAN-GP) to guess the password. The question "How can GAN be used in password guessing" and "How to make the guessing more accurate and efficient" will be answered.

2. Literature Review

2.1 Password guessing

There are three types of user actions that can lead to the creation of insecure passwords:

(1) Users usually use basic terms in their passwords and perform simple string conversions to comply with the site's password creation policy [3].

(2) The repeated use of passwords is common. According to S.Pearman et al. [4] 40% of users reuse their passwords across multiple platforms.

(3) Users prefer to use easy-to-remember passwords, including their personal information.

All of these behaviors can cause user-created passwords to be attacked.

Password guessing tools provide a valuable tool for identifying weak passwords when they are stored in hashed form [21, 22]. The effectiveness of password guessing software relies on the ability to quickly test a large number of highly likely passwords against each password hash. Instead of exhaustively trying all possible character combinations, password guessing tools use words from dictionaries and previous password leaks as candidate passwords. [1]

2.2.1 Password guessing method based on dictionary

Generally speaking, users like to set passwords based on personal information, and studies have shown that at the ports that contain personal information in the order, the user's birthday information is the most common, followed by user name, name, e-mail and so on. By entering these passwords line analysis, from which you can extract some high-frequency words. John the ripper (JTR) [26] is a dictionary-based password cracking tool. To crack a password, you first need to set up a dictionary, and then generate a guess password set according to the dictionary. The hash value of each password is set to locate the target password. In other words, if the target password is not in the password set, it can't be cracked.

2.2.2 Password guessing method based on statistical probability

Generally speaking, the user sets each character in the password from left to right, and the password is the same as the character of the user's mother tongue, the distribution is similar. According to these characteristics, Narayanan [27] applied first-order Markov model to password guessing in 2005, solving the task and reduce the password space to search. Narayanan believes that the adjacent characters in the password are not unique to each other. So the probability of the next character in the password can be calculated by the relationship between adjacent characters, an n-step

horse. The Markov model models the probability of the next character based on the first n characters. The first-order Markov model only considers a single character loses most of the potential password distribution information. In view of this, Tansey [28] proposed a hierarchical horse, the first-order Markov model is extended to the nth-order model, but this method will inflate the model parameters. The expansion results in the decrease of the training speed of the model and the over-fitting of the model. So Ma [29] introduced smoothing technology in 2014. The Markov model is applied to solve the sparsity and over-fitting problems in the high-order Markov model. After that, Duermuth [30] proposed OMEN, an improved Markov model, which can generate the password set in the order of decreasing probabilities., which greatly speeds up the speed of password guessing.

2.2.3 Password guessing method based on deep learning

At present, the method of password guessing based on deep learning has made great progress. And compared with the traditional method of password guessing, it has a larger password space, the generated samples are not limited to training samples, password translation rules and password structure do not need to be artificially designed.

According to the structure of the model, the password guessing method based on deep learning can be divided into the password guessing method based on cyclic neural network (RNN) and the password guessing method based on generating countermeasure network (GAN).

Melicher [31] first applies deep learning to password guessing tasks, and proposes a password guessing method based on RNN. It's called FLA, but FLA was originally used to detect password strength in real time and needs to be deployed in clients and browsers, so the size of the model is compressed. In addition, FLA also limits the types of passwords, resulting in poor generality. After that, Xu [32] proposed a password guessing method based on LSTM, which can capture the correlation between long passwords. Xia [33] proposed a hybrid model PL (PCFG+LSTM) and PR (PCFG+RNN) combining RNN and PCFG. The password cracking rate is improved to some extent.

In addition to migration learning, FLA gets a slight improvement in long password cracking, compressing the size of the model in FLA optimization techniques are not suitable for general password guessing tasks. When generating a password set size of 10^9, FLA got the cracking rate is 17.2%, but FLA uses too few test set passwords, and the final experimental results are not very convincing. On the other hand, the essence of the password guessing task is to generate a burst password set to crack the target password. In terms of the suitability of the task, GAN is obviously more

suitable for generating tasks, so the follow-up work mainly focuses on the password guessing party based on GAN method, and improve it on this basis.[34]

In 2014, Ian Goodfellow, a student at Universite de Montre é al University Yoshua Bengio (winner of the 2019 Turing Award), proposed the generation of confrontation networks, thus opening up one of the hottest research directions in deep learning. From 2014 to 2019, GAN research has made steady progress and good news has spread frequently. The word “GAN” stands for generative adversarial network.

Hitaj [1] applied the GAN to the password guessing task in 2019, and proposed a method based on PassGAN model of WGAN-GP. PassGAN replaces two-dimensional convolution of convolution neural network in WGAN-GP .Instead of one-dimensional convolution, a character-level password generation model is constructed. By confronting the generator and the discriminator in training, PassGAN can generate passwords that are difficult to distinguish between true and false.

The ability of PassGAN to autonomously learn characteristics and patterns constituting a password, much like DeepMinds’ AlphaGo’s ability to autonomously learn the game of Go [5], drew significant attention from several media outlets.

For instance, PassGAN has been reported in articles from Science Magazine [6], The Register [7], Inverse [8], Threatpost [9], Dark Reading [10], Technology Review News [11], Sensors Online [12], and others [13,14,15,16,17,18].

Further, PassGAN was selected by Dark Reading as one of the coolest hacks of 2017 [19].

UC Berkeley has included PassGAN as reading material in their graduate-level course titled Special Topics in Deep Learning [20].

3 . Methodology

In this study, WGAN-GP will be used in guessing password. The theory of original GAN, WGAN, WGAN-GP will be introduced respectively.

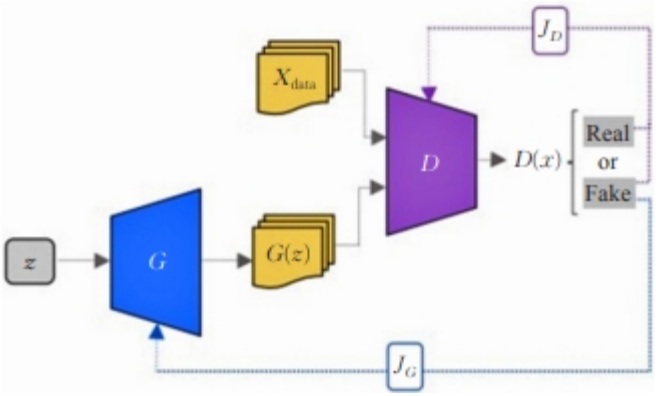
3.1 Theory of original GAN

The so-called "generating adversarial network" is a kind of neural network, which samples from complex probability distribution by training discriminator and generator in turn.

To put it simply, the basic idea of "GAN" is that the generator and the discriminator play a game of "as virtue rises one foot and vice rises ten": the discriminator should practice the "golden eye" and try its best to distinguish between the real sample (such as the real picture) and the false sample generated by the generator; the generator

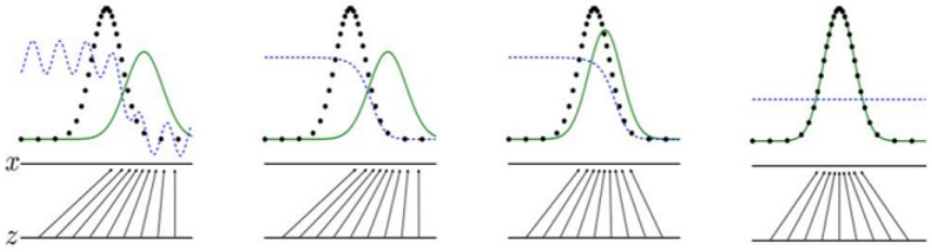
should learn to "confuse the false with the real" and generate the "fake sample" that the discriminator judges as real. The ideal state of competition is for both sides to make continuous progress-the eyes of the discriminator are getting clearer and the deceptive ability of the generator is also increasing.

GAN is a network model similar to two-person game, which consists of generating network G and discriminant network D. The architecture of GAN is shown in figure1. X_{data} and G(z) represent real data samples and pseudo data samples generated by generator G respectively, and discriminator D judges whether the input data is true or false. In GAN, the generator G takes the fixed-length random noise vector z (uniform distribution or Gaussian distribution) as the input. The purpose of the generator is to make the generated data distribution approximate to the real data distribution as far as possible. The input of discriminator D has two parts: X_{data} and G(z). Its output is a probability value, which indicates the probability that D thinks the input data is a real sample, and the output will be fed back to G to guide the training of G. Ideally, D cannot tell whether the input data comes from real data or generated data, that is, the output probability value of D is 1/2 each time, and the model is optimal.（Nash equilibrium）[23]



[23] fig1 architecture of GAN

The original GAN uses two objective functions: (1) D minimizes the negative logarithmic likelihood of binary classification, and (2) G maximizes the probability that the sample is real. D parameter is θ_D, G parameter is θ_G. θ_G and θ_D are minimized and maximized objective function respectively, so this is a zero-sum game. P_r and p_g represent the probability distribution of real data defined in data space x and z defined in hidden space respectively.



a) Initial state b) train discriminator c) train generator d) model convergence

[23]fig2 the training process of GAN

The optimization problem solved by GANs can be summarized as follows：[23]

$$min_{\theta_G} max_{\theta_D} V(G,D) = min_{\theta_G} max_{\theta_D} \{E_{x\sim P_r}[lnD(x)] + E_{x\sim p_g}[\ln(1-D(x)]\}$$

The goal of generator is to minimize $E_{x\sim P_r}[lnD(x)] + E_{x\sim p_g}[\ln(1-D(x)]$

The goal of discriminator is to maximize $E_{x\sim P_r}[lnD(x)] + E_{x\sim p_g}[\ln(1-D(x)]$

From a wider point of view，

$$V(G,D) = E_{x\sim P_r}[lnD(x)] + E_{x\sim p_g}[\ln(1-D(x)]$$

$$= \sum p_r(x)\cdot \ln D(x) + \sum p_g(x)\cdot \ln[1-D(x)]$$

It is a bivariate cross entropy loss function.

Since this is a maximum and minimum game, we might as well fix one object first and find the maximum (minimum) value of another object on this basis. We choose to fix G first and consider the maximum value of D in this case.

We know that given $D(x) = a\ln(x) + b\ln(1-x)$, when $x = \frac{a}{a+b}$, D(x) has a maximized value. (By using derivation we can easily know it.)

So the maximized value of D(x) is $D^* = \frac{p_r(x)}{p_r(x)+p_g(x)}$

Then, putting $D^* = \frac{p_r(x)}{p_r(x)+p_g(x)}$ into $V(G,D)$, we have:

$$\sum p_r(x)\cdot \ln\left(\frac{p_r(x)}{p_r(x)+p_g(x)}\right)+\sum p_g(x)\cdot \ln\left(\frac{p_g(x)}{p_r(x)+p_g(x)}\right)$$

$$= KL(p_r(x)||\frac{p_r(x)+p_g(x)}{2})+KL((p_g(x)||\frac{p_r(x)+p_g(x)}{2})-2ln2$$

$$= 2JS(p_r(x)||p_g(x))-2ln2$$

KL divergence and JS divergence are defined as follows:

$$\mathrm{KL(P||Q)} = \sum p(x)ln\frac{p(x)}{q(x)}$$

$$\mathrm{KL(Q||P)} = \sum q(x)\frac{q(x)}{p(x)}$$

we can infer that

$$\mathrm{KL(P||Q)} \geq 0;$$

$$\mathrm{KL(P||Q)} \neq \mathrm{KL(Q||P)}$$

$$\mathrm{JS(P||Q)} = \frac{1}{2}KL(P||\frac{P+Q}{2})+\frac{1}{2}KL(Q||\frac{P+Q}{2})$$

$$= \frac{1}{2}\sum p(x)\ln\left(\frac{p(x)}{p(x)+q(x)}\right)+\frac{1}{2}\sum q(x)\ln\left(\frac{q(x)}{p(x)+q(x)}\right)+ln2$$

That's because $\sum p(x) = \sum q(x) = 1$.

During the training process, $\mathrm{JS}(p_r||p_g)$ gets smaller, the model gets more accrate.

3.2 Theory of WGAN

The disadvantage of KL divergence and JS divergence is that once there is no overlap between the two probability distributions, KL divergence will be meaningless and JS divergence will become a constant ln2. This makes us hard to use gradient decline to optimize the model. That's why GAN is HARD TO TRAIN!

There is a figure to show the disadvantage of KL and JS divergence.

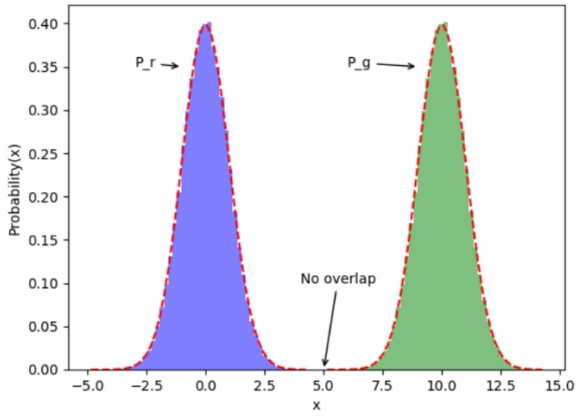


fig3

WGAN uses a new distance definition Wasserstein Distance (bulldozer distance), and theoretically gives the reason for the instability of GAN training, that is, cross entropy (JS divergence) is not suitable to measure the distance between distributions with disjoint parts. Instead, Wassertein distance is used to measure the distance between the generated data distribution and the real data distribution, which theoretically solves the problem of training instability. WGAN uses ADAM as optimizer.

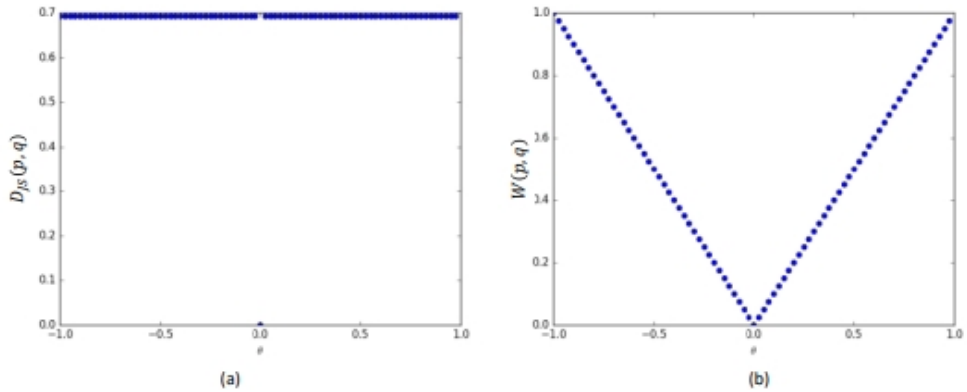


fig4 the difference between Wesserstein distance and JS divergence

The definition of Wasserstein distance can be summarized as follows：

$$W(p,q) = inf_{\gamma \sim \prod(p,q)} E_{(x,y)\sim\gamma}[||x-y||]$$

(||·|| means norm)

To know the theory of WGAN, we should first know what is Lipschitz condition.

If a function f satisfies: $\forall x_1, x_2 \in \mathrm{dom(f)}$,if $|f(x_1) - f(x_2)| \leq K \cdot |x_1 - x_2|$

We call it K- Lipschitz continuous condition.

But it is not possible to actually calculate every (x, y) from $\prod(\mathrm{p,q})$, so we use K-Lipschitz to replace it.

It is: $\mathrm{W(p,q)} = \frac{1}{K} sup_{f\ satisfies\ K-Lipchitz}\{E_{x\sim p_r}[f(x)] - E_{x\sim p_g}[f(x)]\}$

In $\mathrm{W(p,q)} = inf_{\gamma\sim\prod(\mathrm{p,q})} E_{(x,y)\sim\gamma}[||x-y||]$, y just means $x\sim p_g$.

Because of the strong fitting ability of neural networks, we can assume that there is a f that can reach this upper bound.

In most researches, K=1 is the best for training.

Now, we use the function of discriminator $D_\theta(x)$ as f(x).

In other words, it is turned to : $\max\{E_{x\sim p_r}[D_\theta(x,w)] - E_{x\sim p_g}[D_\theta(x,w)]$

Here, w means a parameter in discriminator.

So as it is a function of D, for the real x it should give a high score, for the generated x it should give a low score.

During the training, in order to make the discriminator satisfy the K-Lipschitz continuity condition, a simple method is to limit the parameter gradient in the discriminator to a certain range, and the gradient is generally related to the parameter size, so the K-Lipschitz continuity condition can be achieved through the strategy of weight clipping.

Mode collapse means that the sample generated by the model is single and the diversity is poor. Because the discriminator can only identify whether a single sample is sampled from the real distribution, there is no explicit constraint on the sample diversity, which leads to the possible tilt of the generated model. To generate a small number of high-quality samples in the partial interval of the real distribution, so as to

obtain a higher output of the discriminator probability value. Model collapse is common in GAN.[34]

WGAN found the reason why the original GAN is prone to training instability from the theoretical level, and gave a new distance measurement standard and engineering implementation solution, and achieved good results. WGAN also alleviates the problem of mold collapse to some extent, and the model using WGAN is not prone to model collapse. It should be noted that WGAN (general) can not improve the generation effect of the model, but only ensures the stability of the model training.[34]

WGAN-GP (IWGAN):

But weight clipping may cause gradients to disappear or explode. The reason is to distinguish that the network is a multi-layer network, how to set the clipping threshold is very important, the smaller clipping threshold can easily lead to the disappearance of the gradient. On the contrary, a large clipping threshold can easily lead to gradient explosion, and only if it is set properly can the generator get a return gradient of the appropriate size.

GP stands for gradient penalty. What is output in WGAN is the was distance, the output of the original GAN is within the (0,1) interval, but the Wassertein distance has no upper and lower bounds. It means that as the training goes on, the discriminator will never converge, although weight cutting has been done in reference [25], but this method does not make D is really limited to the Lipschitz function.

It is pointed out that WGAN does not limit D to the Lipschitz function. Clipping will lead to bad behavior of the discriminator, and it is recommended to add a gradient penalty. It's not a weight cut. It shows that the discriminator is guaranteed by weight clipping. The Lipschitz condition limits the discriminator to one of all Lipschitz functions in a very limited subset; this skews the discriminator towards simple functionality. Weight clipping also creates a gradient problem because it pushes the weight to clipping. The extremes of the range. In order to achieve Lipschitz condition by constraining the gradient of the discriminator, it is suggested to add a gradient penalty item, not weight clipping. [24,34]

Gradient penalty limits the size of the gradient by setting an additional loss term:

$$E_{x\sim p_r}[D(x)] - E_{x\sim p_g}[D(x)] - \mu E_{x\sim X}[(||\nabla_{x'} D(x')||_2 - 1)^2]$$

X is the whole sample space, $\nabla_x D(x)$ is the gradient. When the 2-norm of the gradient vector is bigger than 1, it will be penalized.

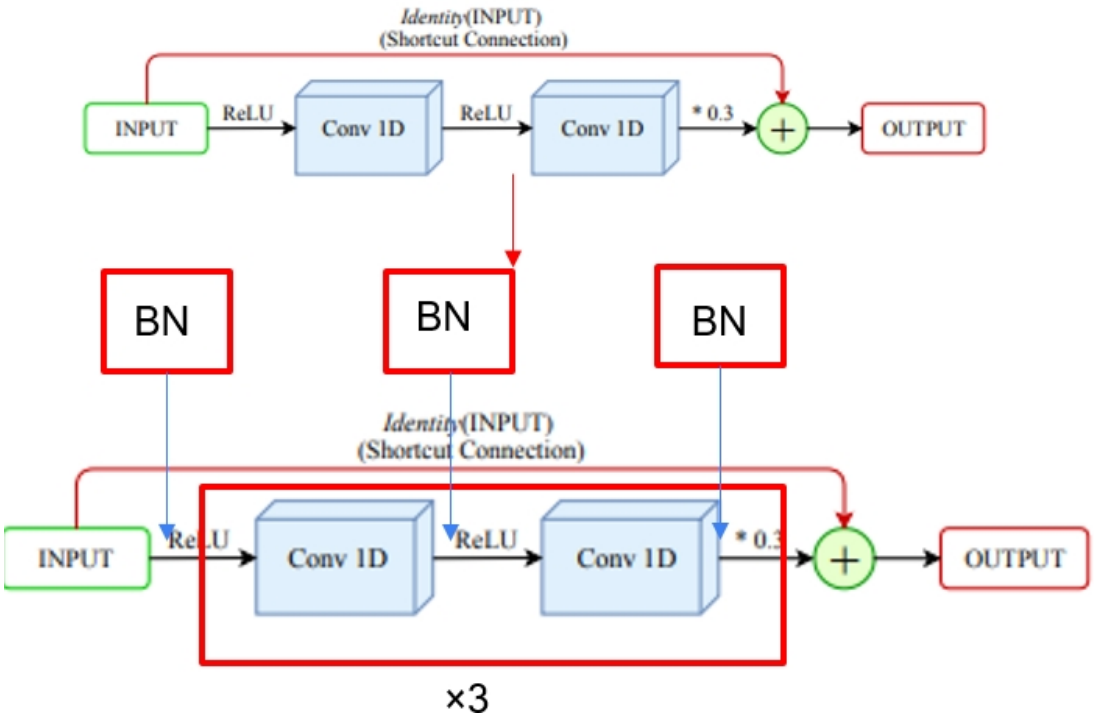
$E_{x\sim X}$ requires the model to sample throughout the sample space and calculate the penalty average. So that's why $x'$is defined as: $x' = \alpha x_r + (1-\alpha)x_g$, α is a random number in range (0,1).

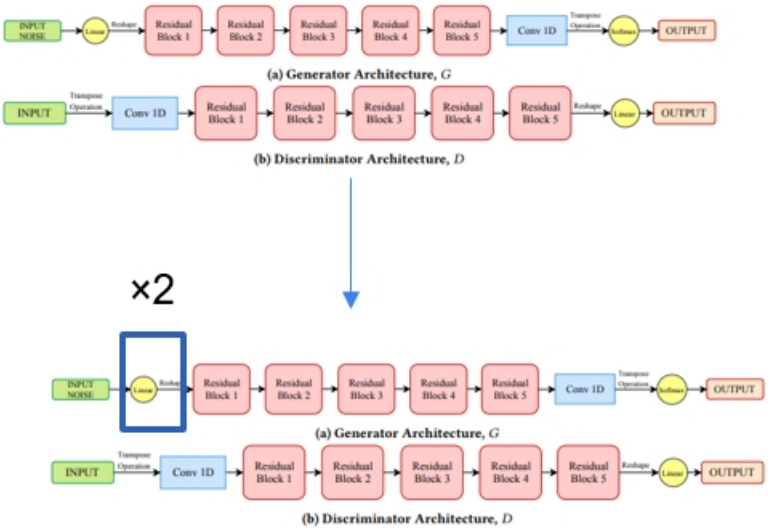
I use WGAN-GP in the experiment.

I use Google Colab to put my code and train my model. The dataset I use for training is rockyou.

3.3 My Approach

Compared with the original PassGAN(using WGAN-GP), I modify the algorithm structure , including adding another linear block to the generator and adding a batch normalization to the residual block in front of each ReLU.



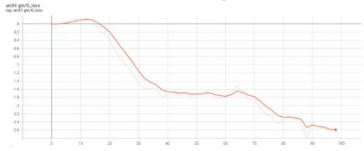


4. Results

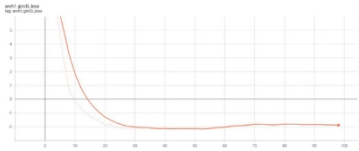
This is the training process:

The training process when epoch=100 and 10000 :
epoch=100:

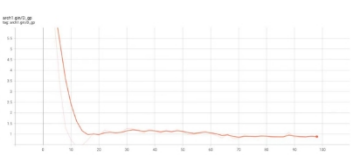
Loss function of generator



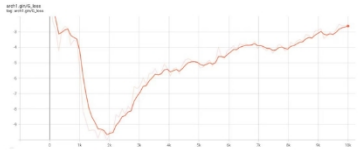
Loss function of discriminator

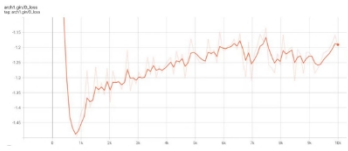


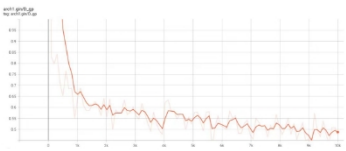
Strength of gradient penalty



epoch=10000:







We can see that during training, the loss function of both D and G get minimized when the epoch (100and10000) is about 20~30 and 1000~2000, so when training PassGAN, we should use the strategy of early stopping.

This is the word cloud of the generated password:



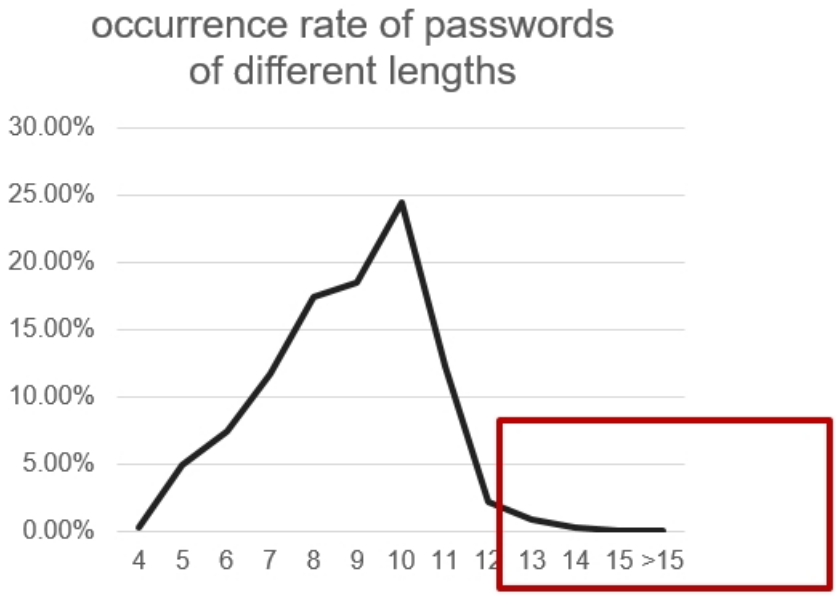
And I carried out a table for comparing the effect of different models. (By using the hit rate on the testing set.)[1]

RockYou testing set

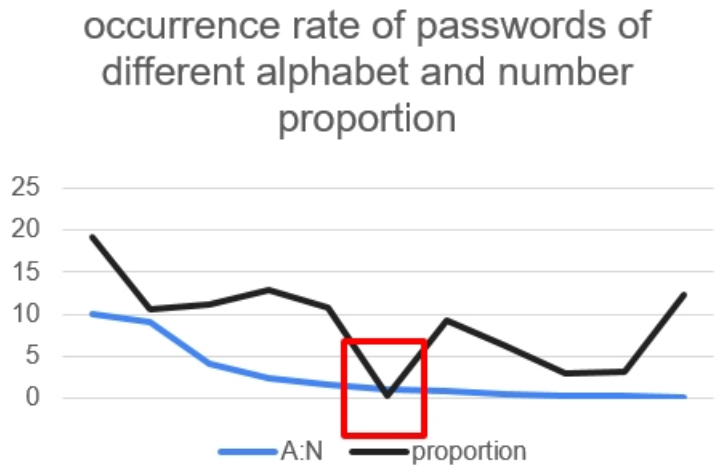
Approach	Unique Passwords	Matches	Number of passwords required for PassGAN to outperform	PassGAN Matches	Our GAN Matches
Markov Model (3-gram)	4.9 x 10^8	532961(26.93%)	2.47 x 10^9	532962(26.93%)	678922(34.76%)
HashCat gen2	10^9	597899(30.22%)	4.8 x 10^9	625245(31.60%)	832802(42.09%)
FLA	7.4 x 10^8	652585(32.99%)	6 x 10^9	653978(33.06%)	869596(43.96%)

5. Discussion

We can see that when the length >15 ,it’s hard for the model to guess it correctly.



We can see that when proportion of alphabets and numbers comes to about 1:1, the hit rate is min.



By optimizing the structure of the algorithm, our GAN has a better performance on the testing set than the original PassGAN(Same in WGAN-GP)

RockYou testing set

Approach	Unique Passwords	Matches	Number of passwords required for PassGAN to outperform	PassGAN Matches	Our GAN Matches
Markov Model (3-gram)	4.9 x 10^8	532961(26.93%)	2.47 x 10^9	532962(26.93%)	678922(34.76%)
HashCat gen2	10^9	597899(30.22%)	4.8 x 10^9	625245(31.60%)	832802(42.09%)
FLA	7.4 x 10^8	652585(32.99%)	6 x 10^9	653978(33.06%)	869596(43.96%)

6. Conclusion

The GAN I used in the model has a good performance in password guessing, compared with formal models(especially the original GAN).

The AI password cracker represented by PassGAN will bring unprecedented action capabilities to hackers and attackers, and the whole security world will probably be changed in the future.

But as users ,we can still secure our passwords now.

The methods are listed below.

First, one way to ensure that passwords are protected against AI attacks is to use at least 15 characters.

Second, when setting a password, you should try to ensure that the ratio of letters to numbers is similar, and add special symbols.

And, from the point of view of probability distribution, passwords should not be reused on multiple platforms.

Also, adding an additional security layer through two-factor or multi-factor authentication is a good method. It avoids the problem that passwords are vulnerable to attacks.

References

[1] BRILAND HITAJ, PAOLO GASTI, GIUSEPPE ATENIESE, et al. PassGAN: A Deep Learning Approach for Password Guessing[C]. //Applied Cryptography and Network Security: 17th International Conference, ACNS 2019, Bogota, Colombia, June 5-7, 2019, Proceedings.:Springer, 2019:217-237.
[2]J. Bonneau, C. Herley, P. C. Van Oorschot, and F. Stajano, “The quest to replace passwords: A framework for comparative evaluation of web authentication schemes,” in 2012 IEEE Symposium on Security and Privacy. IEEE, 2012, pp. 553–567.
[3] A. Narayanan and V. Shmatikov, “Fast dictionary attacks on passwords using time-space tradeoff,” in Proceedings of the 12th ACM Conference on Computer and Communications Security, 2005, pp. 364–372.
[4] S. Pearman, J. Thomas, P. E. Naeini, H. Habib, L. Bauer, N. Christin, L. F. Cranor, S. Egelman, and A. Forget, “Let’s go in for a closer look: Observing passwords in their natural habitat,” in Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 295–310.
[5] alphago 2018. AlphaGo – DeepMind. https://deepmind.com/research/alphago/.
[6] Matthew Hutson. 2017. Artificial intelligence just made guessing your password a whole lot easier. Science (2017). http://www.sciencemag.org/news/2017/09/
[7] Iain Thomson. 2017. AI slurps, learns millions of passwords to work out which ones you may use next. The Register (2017). https://www.theregister.co.uk/2017/09/20/researchers_train_ai_bots_to_crack_passwords/
[8] Allee Manning. 2017. Researchers Show How A.I. Is the End of Passwords as We Know Them. Inverse (2017). https://www.inverse.com/article/36604-ai-cracking-passwords
[9] Michael Mimoso. 2017. Deep-Learning PassGAN Tool Improves Password Guessing. Threatpost (2017). https://threatpost.com/deep-learning-passgan-tool-improves-password-guessing/128039/
[10] Jai Vijayan. 2017. PassGAN: Password Cracking Using Machine Learning. Dark Reading (2017). https://www.darkreading.com/analytic
[11] Jamie Condliffe. 2017. A Pair of AIs Have Become VeryGood at Guessing Your Passwords. Technology Review News (2017).https://www.technologyreview.com/the-download/608897/a-pair-of-ais-have-become-very-good-at-guessing-your-passwords/
[12] Mathew Dirjish. 2017. Artificial Intelligence, Slayer of Passwords.Sensors Online (2017). https://www.sensorsmag.com/embedded/artificial-intelligence-slayer-passwords
[13] Michael Cobb. 2017. How machine learning-powered password guessing impacts security. TechTarget (2017). http://searchsecurity.techtarget.com/tip/How-machine-learning-powered-password-guessing-impacts-security
[14] J. Wolfgang Goerlich. 2017. PassGAN for AI Password Guessing. Stuck in Traffic (2017). https://www.youtube.com/watch?v=b92sTdRRwvs

[15] Elisabetta Intini. 2017. Líntelligenza artificiale indovina le password. Focus IT (2017). https://www.focus.it/tecnologia/digital-life/lintelligenza-artificiale-indovina-le-password
[16] Adam Janofsky. 2018. AI Could Make Cyberattacks More Dangerous, Harder to Detect. The Wall Street Journal (2018). https://www.wsj.com/articles/ai-could-make-cyberattacks-more-dangerous-harder-to-detect-1542128667
[17] Isaac Madan. 2017. Up to Speed on AI & Deep Learning: October Update. Hackernoon (2017). https://hackernoon.com/up-to-speed-on-deep-learning-october-update-815f5eef0e2b
[18] 2017. These Powerful New Password Crackers Challenge Our Notions of 'Secure'.Stevens Institute of Technology (2017). https://goo.gl/qcwNVd
[19] Kelly Jackson Higgins. 2017. The Coolest Hacks of 2017. Dark Reading (2017). https://www.darkreading.com/threat-intelligence/the-coolest-hacks-of-2017/d/d-id/1330699
[20] 2017. Arxiv Summaries Week 09/11 – PassGAN: A Deep Learning Approach for Password Guessing. CS 294-131:Special Topics in Deep Learning (2017). https://berkeley-deep-learnieng.github.io/cs294-131-f17/arxiv.html
[21] Colin Percival and Simon Josefsson. 2016. The scrypt password-based key derivation function. Technical Report.
[22] Niels Provos and David Mazieres. 1999. Bcrypt algorithm. In USENIX
[23]孙书魁,范菁,曲金帅&路佩东.(2022).生成式对抗网络研究综述.计算机工程与应用(18), 90-103.
[24] QI G J.Loss- sensitive generative adversarial networks on lipschitz densities[J].International Journal of Computer Vision， 2020，128（5）：1118-1140.
[25] ARJOVSKY M，CHINTALAS，BOTTOU L.Wasserstein generative adversarial networks[C]//Proceedings of the 34th International Conference on Machine Learning， 2017：214-223.
[26] Wikipedia contributors. John the Ripper. In Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=John_the_Ripper&oldid=1070869618, 2022. [accessed 31-March-2022].
[27] Narayanan A, Shmatikov V. Fast dictionary attacks on passwords using time-space tradeoff[C]. Proceedings of the 12th ACM conference on Computer and communications security. 2005: 364-372.
[28] Tansey W. Improved models for password guessing[J]. University of Texas, Tech. Rep, 2011.
[29] Ma J, Yang W, Luo M, et al. A study of probabilistic password models[C]. 2014 IEEE Symposium on Security and Privacy. IEEE, 2014: 689-704.
[30] Dürmuth M, Angelstorf F, Castelluccia C, et al. OMEN: Faster password guessing using an ordered markov enumerator[C]. International symposium on engineering secure software and systems. Springer, Cham, 2015: 119-132.
[31] Melicher W, Ur B, Segreti S M, et al. Fast, lean, and accurate: Modeling password guessability using neural networks[C]. 25th USENIX Security Symposium (USENIX Security 16). 2016: 175-191.

[32] Xu L, Ge C, Qiu W, et al. Password guessing based on LSTM recurrent neural networks[C]. 2017 IEEE International Conference on Computational Science and Engineering (CSE) and IEEE International Conference on Embedded and Ubiquitous Computing (EUC). IEEE, 2017, 1: 785-788.
[33] XIA Z, YI P. Multi-Source Deep Learning Model for Password Guessing[J]. Communications Technology, 2019.
[34]周涛. 基于深度学习的口令猜解方法研究[D]. 广东:华南理工大学,2022.